

Piezo-LLM — Project Handbook

Piezoelectric Molecular Crystals: ML-potential MD simulations + RAG knowledge platform IIT Delhi · Department of Chemistry

Last updated: 18 July 2026 Repository: <https://github.com/Him3shRana/Piezo-LLM-IIT-Delhi>

Table of contents

1. [Project overview](#)
 2. [HPC infrastructure](#)
 3. [Python environments](#)
 4. [ML interatomic potentials](#)
 5. [Model cache locations](#)
 6. [Simulation scripts](#)
 7. [MD parameters explained](#)
 8. [Critical fixes — institutional knowledge](#)
 9. [Known issues and open problems](#)
 10. [Data layout and schema](#)
 11. [Frontend \(GUI\)](#)
 12. [Backend \(Flask API\)](#)
 13. [LLM / RAG pipeline](#)
 14. [Git repository hygiene](#)
 15. [Roadmap](#)
 16. [Pending checklist](#)
 17. [Command cheat sheet](#)
 18. [Troubleshooting table](#)
-

1. Project overview

Goal

Benchmark modern machine-learning interatomic potentials (MLIPs) on **piezoelectric molecular crystals** — determining which model best reproduces experimental structure and density — and expose the whole corpus (crystals, papers, simulations) through a searchable, LLM-assisted interface.

Three components

Component	What it does	Stack
Simulation pipeline	NPT/NVT MD on ~48 molecular crystals across 15+ MLIPs	Python, ASE, PBS/Torque, A100 GPU
Knowledge base	Crystal structures, CIFs, papers, derived properties	JSON + Chroma vector DB
GUI + RAG chat	Browse crystals, view 3D structures, ask questions	React/Vite + Flask + Qwen3-8B

Scale as of now

- 48 crystals in master_database.json
- 63 reference papers via /api/papers
- 15+ MLIPs installed across 8 conda/venv environments
- 2 MD engines wired: ASE (primary), LAMMPS (secondary)

 **Known inconsistency:** the Dashboard hardcodes 36 for "Research Papers" while the Papers page reports 63 from the API. Fix the Dashboard to read the real count.

2. HPC infrastructure

Cluster

Item	Value
Institution	IIT Delhi
Scheduler	PBS / Torque
GPU	NVIDIA A100-SXM4-80GB
Cluster CUDA	12.6
Username	cyz218376
Login host	pragya.iitd.ac.in
Login nodes seen	login01, login03
Compute nodes seen	amilan001 ... amilan011

Key paths

```

/home/chemistry/phd/cyz218376/
├─ home/software/                # all virtualenvs live here
│   ├─ chgnet_env/
│   ├─ grace_env/
│   ├─ mace-gpu/
│   ├─ mace_mp0_env/
│   ├─ orb_env/
│   ├─ mattersim_env/
│   ├─ uma_env/
│   └─ upet_env/
├─ himesh_work/
│   ├─ data/PMC-XXX/*.cif        # input structures
│   ├─ mace_models/             # manually downloaded .model files
│   └─ testing/                 # scripts + runs live here
│       ├─ all_npt_ase.py
│       ├─ all_nvt_ase.py
│       └─ runs/PMC-XXX/ase-npt/<model>-<version>/<T>K_<P>bar_<N>x<N>x<N>/
└─ .cache/                      # auto-managed model caches

```

● Critical constraint: compute nodes have NO internet

This bites constantly. Any model that auto-downloads **must be fetched on a login node first**, otherwise you get:

```

urllib.error.URLError: <urlopen error Tunnel connection failed: 302 Moved
Temporarily>
ProxyError('Unable to connect to proxy', ...)
httpx.ConnectError: [Errno 101] Network is unreachable

```

Rule: download on login node → run on compute node.

For HuggingFace-backed models (UMA, PET) also export offline mode on the compute node:

```

bash

export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1

```

Do **not** bake this into the script — it would block legitimate downloads on the login node. Set it per-session or in the PBS job script.

Output directory convention

```

runs/<PMC-ID>/ase-npt/<family>-<version>/<T>K_<P>bar_<N>x<N>x<N>/
├─ config.yaml                # full run settings + timestamp
├─ .run.lock                  # PID lock, prevents double-runs
├─ minimization/
│   ├─ restart.extxyz
│   ├─ state.json
│   ├─ structure.pdb
│   └─ minimization.log
├─ equilibration/
│   ├─ restart.extxyz        # checkpoint, written every 500 steps
│   ├─ state.json            # {last_completed_step, target_step, status,
timestamp}
│   ├─ trajectory.pdb        # appended every 1000 steps
│   ├─ structure.pdb         # final frame
│   ├─ equilibration.log     # timestamped step lines
│   └─ temp_vs_time.png
└─ production/
    ├─ restart.extxyz
    ├─ state.json
    ├─ trajectory.pdb
    ├─ traj.extxyz           # ← main analysis trajectory (RDF etc.)
    ├─ structure.pdb
    └─ production.log

```

3. Python environments

Each model family needs its own environment because of conflicting CUDA / torch / TF requirements.

Environment	Module stack	Models	Status
<code>chgnet_env</code>	<code>cuda/12.6</code> <code>gcc/10.5.0</code> <code>conda-2025/pytorch-gpu-2.9.1/py3.14</code>	CHGNet	✅ working
<code>grace_env</code>	<code>cuda/12.6</code> <code>gcc/10.5.0</code> <code>conda-2025/tf-gpu-2.19.1/py3.12</code>	GRACE-2L-OAM, GRACE-2L-OMAT-medium-ft-AM	✅ working
<code>mace-gpu</code>	<code>conda-2025/pytorch-gpu-2.9.1/py3.14</code>	MACE-OFF23 (S/M/L), PolarMACE (S/M/L)	⚠️ torch built for CUDA 13.0 , cluster has 12.6
<code>mace_mp0_env</code>	<code>cuda/12.6</code> <code>gcc/10.5.0</code> <code>conda-2025/pytorch-gpu-2.9.1/py3.14</code> + torch cu126	MACE-MP-0, MACE-MPA-0, MACE-MH-0, MACE-MH-1	✅ working
<code>orb_env</code>	<code>cuda/12.6</code> <code>gcc/10.5.0</code> <code>conda-2025/tf-gpu-2.19.1/py3.12</code>	ORB v0.7.0 (all variants)	✅ working
<code>mattersim_env</code>	<code>cuda/12.6</code> <code>gcc/10.5.0</code> <code>conda-2025/tf-gpu-2.19.1/py3.12</code>	MatterSim v1.2.5 (5M / 1M)	✅ working
<code>uma_env</code>	<code>conda-2025/...</code> + <code>fairchem-core</code>	UMA-S-1p1, UMA-M-1p1	✅ working (HuggingFace auth required)
<code>upet_env</code>	<code>cuda/12.6</code> <code>gcc/10.5.0</code> <code>conda-2025/tf-gpu-2.19.1/py3.12</code>	PET-OAM-XL	⚠️ needs <code>HF_HUB_OFFLINE=1</code> on compute node

Why `mace_mp0_env` exists separately

`mace-gpu` ships torch `(2.11.0+cu130)`. MACE-MPA-0 uses TorchScript JIT, which tries to compile CUDA kernels at runtime and fails:

```
RuntimeError: nVRTC: error: failed to open libnVRTC-builtins.so.13.0
```

`mace_mp0_env` was created with torch built for **cu126** to match the cluster:

```
bash
```

```
module load cuda/12.6 gcc/10.5.0 conda-2025/pytorch-gpu-2.9.1/py3.14
python3 -m venv ~/home/software/mace_mp0_env
source ~/home/software/mace_mp0_env/bin/activate
pip install --upgrade pip
pip install torch torchvision torchaudio --index-url https://download.pytorch.o
pip install mace-torch ase pymatgen
```

Environment creation pattern (for future models)

```
bash
```

```
module load cuda/12.6
module load gcc/10.5.0
module load conda-2025/tf-gpu-2.19.1/py3.12          # or pytorch-gpu-2.9.1/py3
python3 -m venv ~/home/software/<name>_env --system-site-packages
source ~/home/software/<name>_env/bin/activate
pip install --upgrade pip setuptools wheel
pip install torch torchvision torchaudio
pip install <package>
```

Exit an environment with `deactivate`.

4. ML interatomic potentials

Full inventory

Model	Training data	DFT level	Params	Good for molecular crystals?
CHGNet	MPtrj (1.58M)	PBE+U	412 K	⚠️ General inorganic
MACE-OFF23 (S/M/L)	Organic molecules	ω B97X-D	—	✅ Best for organic
PolarMACE (S/M/L)	Polar organic molecules	ω B97M-D3	—	✅ Good for polar
MACE-MP-0 (S/M/L)	MPtrj	PBE (VASP)	—	⚠️ Only 1.8% organic
MACE-MPA-0	MPtrj + Alexandria	PBE	—	⚠️ Better than MP-0
MACE-MH-0 / MH-1	Multi-head, multiple datasets	mixed	—	✅ with <code>spice_wB97M</code> head
GRACE-2L-OAM	OMat24 + MPtrj + sAlex	PBE+U	—	⚠️ General
GRACE-2L-OMAT-medium-ft-AM	OMat24 fine-tuned	PBE+U	—	⚠️ General
ORB-v2	MPtrj + Alexandria	PBE	—	⚠️ direct model — no stress
ORB-v3-conservative-inf-omat	OMat24	PBE+U	—	✅ has stress, NPT-capable
ORB-v3-direct-inf-omat	OMat24	PBE+U	—	⚠️ direct — no stress
MatterSim-v1.0.0-5M	Proprietary (Microsoft)	PBE	4.55 M	⚠️ validated on mol. crystals
UMA-S-1p1 / UMA-M-1p1	OMat24+OC20+ODAC25+OMC25+OMol25 (569M)	PBE+U	150M / 1.4B	✅ only model trained on molecular

PET-OAM-XL OMat24 + MPtrj + sAlex

PBE+U —

⚠ General

Downloaded but NOT yet integrated

Present in `~/ .cache/huggingface/hub/models--facebook--OMAT24/`:

- `eqv2_86M_omat_mp_salex.pt` — EquiformerV2
- `esen_30m_oam.pt` — eSEN-30M-OAM

Also on the wishlist: **NequIP-OAM-XL**.

Recommended models for this project

- UMA-M-1p1** — the only model with molecular crystals (OMC25, 27M structures) in training
- MACE-OFF23 medium** — organic-specific, ω B97X-D includes dispersion
- MACE-MH-1 (spice head)** — SPICE data, ω B97M-D3
- ORB-v3-conservative-inf-omat** — conservative forces, stable NPT

MACE-MH-1 heads

MACE-MH-1 is multi-head. Passing no `head` raises an error listing the options:

```
['matpes_r2scan', 'mp_pbe_refit_add', 'spice_wB97M', 'oc20_usempbbe', 'omol', 'omat_pbe']
```

Head	Use for
<code>spice_wB97M</code>	✔ organic molecules — use this
<code>omat_pbe</code>	inorganic materials
<code>mp_pbe_refit_add</code>	Materials Project style
<code>matpes_r2scan</code>	r ² SCAN functional
<code>omol</code>	molecules
<code>oc20_usempbbe</code>	catalysis

UMA task names

```
FAIRChemCalculator(predictor, task_name=...):
```


Task	Domain
<code>omc</code>	✔ molecular crystals — use this
<code>omol</code>	molecules
<code>omat</code>	inorganic materials
<code>oc20</code>	catalysis
<code>odac</code>	MOFs / direct air capture

On DFT levels — why it matters

The DFT functional was used to generate the *training data*; you never set it yourself. But it determines what the model inherits:

- **PBE has no dispersion (van der Waals)**. Molecular crystals are held together by vdW forces → PBE-trained models overestimate cell volume by ~1-3% and get intermolecular distances wrong.
- **ωB97X-D / ωB97M-D3** include dispersion → much better for molecular crystals.

Mitigation for MACE-MP-0: enable D3 correction.

```
python
mace_mp(model=..., device=device, default_dtype="float64", dispersion=True)
```

5. Model cache locations

Library	Cache path	Contents
MACE	<code>~/.cache/mace/</code>	<code>MACE-OFF23_{small,medium,large}.model</code> , <code>MACEPOLAR1Mmodel</code> , <code>20231203mace128L1_epoch199model</code> (MP-0 medium), <code>macempa0mediummodel</code> , <code>macemh1model</code>
GRACE	<code>~/.cache/grace/</code>	<code>GRACE-2L-OAM/</code> , <code>GRACE-2L-OMAT-medium-ft-AM/</code>
ORB	<code>~/.cache/</code> <code>cached_path/</code>	2 files, ~97 MB each (hash-named)
MatterSim	<code>~/.local/mattersim/</code> <code>pretrained_models/</code>	<code>mattersim-v1.0.0-5M.pth</code> (87 MB)
UMA	<code>~/.cache/fairchem/</code> <code>models--facebook--UMA/</code>	<code>uma-s-1p1.pt</code> , <code>uma-m-1p1.pt</code>
PET	<code>~/.cache/upet/</code>	<code>pet-oam-xl-v1.0.0.pt</code>
HuggingFace	<code>~/.cache/</code> <code>huggingface/hub/</code>	OMAT24 models, upet snapshots
CHGNet	inside the venv package	auto-managed
Manual MACE	<code>~/himesh_work/</code> <code>mace_models/</code>	<code>.model</code> files downloaded with <code>wget</code>

Caches are shared across environments — `~/.cache` lives in your home directory, so a model downloaded in `orb_env` is visible from `chgnet_env` too. Download once, use anywhere.

6. Simulation scripts

`all_npt_ase.py` — NPT pipeline

Three stages: **minimization** → **equilibration** → **production**.

Arguments

Flag	Default	Meaning
<code>--pmc</code>	<i>required</i>	PMC ID, e.g. <code>PMC-001</code>
<code>--model</code>	<code>mace-off23</code>	model family
<code>--version</code>	<code>medium</code>	model variant / head
<code>--temperature</code>	<i>required</i>	target T in K
<code>--pressure</code>	<code>1.0</code>	target P in bar
<code>--timestep</code>	<code>0.5</code>	fs
<code>--supercell-size</code>	<code>2</code>	builds N×N×N
<code>--minimize-steps</code>	<code>500</code>	LBFGS max steps
<code>--equil-step</code>	—	equilibration target (steps)
<code>--equil-ps</code>	—	equilibration target (ps) — alternative
<code>--target-step</code>	—	production target (steps)
<code>--target-ps</code>	—	production target (ps) — alternative
<code>--checkpoint-every</code>	<code>500</code>	restart.extxyz + state.json write interval
<code>--dump-every</code>	<code>1000</code>	trajectory.pdb / traj.extxyz frame interval
<code>--cif-dir</code>	<code>~/himesh_work/ data</code>	input structures
<code>--runs-dir</code>	<code>runs</code>	output root
<code>--gpu</code>	off	use CUDA
<code>--fresh</code>	off	discard checkpoints (requires <code>--confirm-fresh</code>)
<code>--confirm-fresh</code>	off	safety confirmation for <code>--fresh</code>
<code>--list-models</code>	—	print the model registry and exit

Features

- **Checkpoint / resume** — killed jobs resume from the last `state.json` automatically
- **Skip logic** — if `last_completed_step >= target_step`, the stage is skipped (data safe)
- **ETA + s/step** printed on every progress line
- **Log files** per stage with UTC timestamps
- `calculate_total_time()` — sums wall time across job restarts, ignoring gaps > 10 min
- **Time summary** at pipeline end (equilibration / production / grand total / speed)
- **PID lock** (`.run.lock`) prevents accidental double-runs
- `config.yaml` records every setting plus run start time

Resume behaviour

Scenario	Result	Data safe?
Run 200k, then run 20k	Skipped (already past target)	✓
Run 20k, then run 200k	Resumes 20k → 200k	✓
Same target twice	Skipped	✓
Run with <code>--fresh --confirm-fresh</code>	Restarts, overwrites	✗

`--fresh` deletes only `state.json`; the actual overwrite happens as the new run writes files. **Back up before using it:**

```
bash
```

```
cp -r runs/PMC-001/ase-npt/<model>/300K_1bar_2x2x2/ runs/..._backup/
```

`all_nvt_ase.py` — NVT pipeline

Same model registry, same features, Langevin thermostat, fixed cell. Output goes to `ase-nvt/` instead of `ase-npt/`.

Model registry entry format

python

```
MODELS = {  
    # local .model file  
    ("mace-off23", "medium"): {"family": "mace-off23", "size": "medium",  
                                "model_path": f"{MODELS_DIR}/MACE-OFF23_medium.m  
                                "calc_type": "mace_off"},  
  
    # auto-managed cache  
    ("chgnet", "pretrained"): {"family": "chgnet", "size": "pretrained",  
                                "calc_type": "chgnet"},  
  
    # named model in library registry  
    ("grace", "GRACE-2L-OAM"): {"family": "grace", "size": "GRACE-2L-OAM",  
                                "model_name": "GRACE-2L-OAM", "calc_type": "gra  
  
    # multi-head  
    ("mace-mh1", "spice"): {"family": "mace-mh1", "size": "spice",  
                             "head": "spice_wB97M", "calc_type": "mace_mh1"},  
}
```

build_calculator() — per-model construction

python

MACE-OFF23

mace_off(model=model_path, device=device, default_dtype="float64")

PolarMACE

MACECalculator(model_paths=model_path, device=device, default_dtype="float64")

CHGNet (CHGNetASECalc MUST be defined at module level – see §8)

chgnet_model = CHGNet.load()

if device == "cuda": chgnet_model = chgnet_model.to("cuda")

CHGNetASECalc(chgnet_model)

GRACE

TPCalculator(os.path.expanduser(f"~/.cache/grace/{model['model_name']}"))

ORB (compile=False is REQUIRED – see §8)

model_fn = pretrained.ORB_PRETRAINED_MODELS[model["model_name"]]

orb_model, adapter = model_fn(device=device, compile=False)

ORBCalculator(orb_model, adapter, device=device)

MACE-MP-0

mace_mp(model=model["size"], device=device, default_dtype="float64")

MACE-MPA-0 (float32 – float64 crashes, see §8)

mace_mp(model="medium-mpa-0", device=device, default_dtype="float32")

MACE-MH-0 / MH-1

mace_mp(model="mh-1", device=device, default_dtype="float32", head=head)

MatterSim

potential = Potential.from_checkpoint(load_path=model["model_name"], device=device)

MatterSimCalculator(potential=potential, device=device)

UMA

predictor = pretrained_mlip.get_predict_unit(model["model_name"], device=device)

FAIRChemCalculator(predictor, task_name="omc")

PET

UPETCalculator(model=model["model_name"], version="1.0.0",

checkpoint_path=os.path.expanduser("~/.cache/upet/pet-oam-x1-v1."),

device=device)

7. MD parameters explained

Current settings in `run_npt_md_stage()`

```
python

taut = 100 * units.fs          # thermostat coupling time
taup = 800 * units.fs          # barostat coupling time
compressibility_au = 8e-5 / units.GPa
```

What each does

Parameter	Controls	Smaller value	Larger value
<code>taut</code>	how fast T reaches target	faster, more aggressive	slower, gentler
<code>taup</code>	how fast V reaches target P	volume equilibrates faster	volume equilibrates slower
<code>compressibility_au</code>	how much the box rescales per step	slow, stable	fast, risks oscillation

None of these change the **final equilibrium volume** — only how fast you get there.

`units.fs` — what it actually is

A conversion constant from ASE, nothing to do with the model:

```
python

from ase import units
units.fs          # 0.09822694788...  fs → ASE time units
units.GPa         # 0.00623...       GPa → eV/Å³
units.eV          # 1.0              (ASE native)
units.Angstrom    # 1.0              (ASE native)
```

Always write `100 * units.fs`, never bare `100`.

float32 vs float64

	float32	float64
Precision	~7 decimals	~15 decimals
Speed	✓ ~2× faster	slower
Error contributed	~0.001 meV/atom	negligible

Context: ML model error vs DFT is ~30–50 meV/atom. float32 adds ~0.003% on top of that — invisible. Use float32 for MD; float64 only matters for tight geometry optimisation.

Observed: MACE-MPA-0 went **0.81 s/step (float64)** → **~0.097 s/step (float32)**.

Recommended settings for molecular crystals

```
bash

--timestep 0.5          # H atoms are light; 1.0 fs risks instability
--equil-step 200000     # 100 ps minimum; volume drifts for a long time
--target-step 2000000   # 1 ns production for decent statistics
--supercell-size 2      # 2×2×2
```

⚠ Honest caveat: these are reasonable defaults, **not universally optimal**. Each ML potential returns stress on a different scale (see §9), so `taup` and `compressibility` may need per-model tuning. Validate against experimental density before trusting any run.

Validation script

```
python

from ase.io import read
from ase import units
import numpy as np

frames = read("production/traj.extxyz", index=":")
last = frames[-500:]          # last 25% only

volumes = [f.get_volume() for f in last]
mass = last[0].get_masses().sum()          # amu
density = (mass * 1.66054e-27) / (np.mean(volumes) * 1e-30) / 1000
print(f"Simulated density: {density:.3f} g/cm³")

temps = [2.0 * f.get_kinetic_energy() / (3.0 * len(f) * units.kB) for f in last]
print(f"T = {np.mean(temps):.1f} ± {np.std(temps):.1f} K")
print(f"V = {np.mean(volumes):.2f} ± {np.std(volumes):.2f} Å³ "
      f"({np.std(volumes)/np.mean(volumes)*100:.2f}% fluctuation)")
```

Good simulation: T fluctuates ±10–20 K, V stabilises and fluctuates rather than drifting, density within ~5% of experiment.

8. Critical fixes — institutional knowledge

These cost real debugging time. Do not regress them.

1. `CHGNetASECalc` must be at module level

Defining the wrapper class **inside** `build_calculator()` lets Python garbage-collect it after the function returns → `RuntimeError: Atoms object has no calculator`.


```
python

# ✅ module level, wrapped in try/except so other envs don't break
try:
    from ase.calculators.calculator import Calculator
    from pymatgen.io.ase import AseAtomsAdaptor
    class CHGNetASECalc(Calculator):
        implemented_properties = ['energy', 'forces', 'stress']
        ...
except ImportError:
    CHGNetASECalc = None
```

The `try/except` matters — MACE-only environments lack `pymatgen` and would crash on import otherwise.

Also watch for **shadowing**: a leftover duplicate class definition inside the function body causes `UnboundLocalError: cannot access local variable 'CHGNetASECalc'` even though the global exists.

2. Remove the `_safe_refresh` patch — it doubled every force calculation

```
python

# ❌ DELETE THIS — calls get_forces() a second time per step
def _safe_refresh():
    if dyn.atoms.calc is None:
        dyn.atoms.calc = _calc
    dyn.atoms.get_forces()
dyn._refresh_properties = _safe_refresh
```

Impact measured on CHGNet: **8.526 s/step** → **~0.14 s/step**. On MACE-MPA-0: 0.116 → 0.097 s/step.

Keep only:

```
python

_calc = atoms.calc
dyn = NPTBerendsen(...)
dyn.atoms.calc = _calc
```

It also interfered with NPTBerendsen's box rescaling — volume drift dropped from +3.4 Å³/5000 steps to +0.08 Å³/5000 steps once removed.

3. Rebuild the calculator on every resume

```
python
```

```
if action == "resume":  
    atoms = read(restart)  
    atoms.set_pbc(True)  
    calc = build_calculator(model, device=device) # ← rebuild, don't reuse  
    atoms.calc = calc
```

Applies to all three stages, on both skip and resume paths.

4. ORB requires `compile=False`

Without it, TorchInductor fails:

```
torch._inductor.exc.InductorError: IndexError: list index out of range
```

```
python
```

```
orb_model, adapter = model_fn(device=device, compile=False)
```

5. MACE-MPA-0 requires float32

`float64` triggers a CUDA JIT failure in the `mace-gpu` env:

```
RuntimeError: nvrtc: error: failed to open libnvrtc-builtins.so.13.0
```

Root cause is the CUDA 13.0 / 12.6 mismatch. Two mitigations: use `float32`, and run from `mace_mp0_env` (built for cu126).

6. `trajectory.pdb` fresh-run detection

`write(..., append=True)` mixes old and new frames after a fresh restart, producing a trajectory that jumps discontinuously mid-file — which silently corrupts RDF and density analysis.

```
python
```

```
if step % dump_every == 0:  
    pdb_path = os.path.join(stage_dir, "trajectory.pdb")  
    if step == dump_every and start_step == 0: # first write of a fresh ru  
        if os.path.exists(pdb_path):  
            os.remove(pdb_path)  
    write(pdb_path, atoms, format="proteindatabank", append=True)
```

`traj.extxyz` is safe — it's written whole at the end and always overwrites.

7. Download models on the login node

Compute nodes have no internet. See §2.

8. PET / UMA need offline mode on compute nodes

```
bash

export HF_HUB_OFFLINE=1
```

Set per-session, not in the script.

9. Stale `.run.lock`

If a job is killed hard, the lock may survive:

```
Another run is already in progress (PID 12345, lock: .../.run.lock)
```

The script checks `/proc/<pid>` and tolerates a dead PID, but if it does block, delete the file manually.

9. Known issues and open problems

● ORB direct models have no stress → NPT volume frozen

In `ORBCalculator.__init__`:

```
python

self.implemented_properties = model.properties
if self.conservative:
    self.implemented_properties.append("forces")
    if self.model.has_stress:
        self.implemented_properties.append("stress")
```

Direct models (`orb-v2`, `orb-v3-direct-*`) never register `stress`. NPTBerendsen has nothing to act on, so volume stays **exactly** constant — observed at $V = 1879.3936 \text{ \AA}^3$ for 65,000 steps.

Model	NVT	NPT
<code>orb-v2</code>	✓	✗ volume frozen
<code>orb-v3-direct-inf-omat</code>	✓	✗ volume frozen
<code>orb-v3-conservative-inf-omat</code>	✓	✓

● ORB and MatterSim return very large stress

Measured on the 2×2×2 glycine supercell:

```
ORB v3-cons: [-0.300, -0.300, -0.191, ...] eV/Å³ → ~48 GPa
MatterSim-5M: [-0.313, -0.313, -0.194, ...] eV/Å³ → ~44 GPa
```

That's an enormous internal pressure, so the barostat can't reach 1 bar with default coupling. An attempted per-model fix (`taup=100 fs`, `compressibility=1e-3`) was **reverted** — it slowed MatterSim significantly and made MACE volumes expand uncontrollably.

Status: unresolved. Options not yet tried:

- verify the sign/unit convention each library uses for stress
- check whether these are absolute or reference-subtracted stresses
- per-model barostat parameters, applied carefully and benchmarked

● Volume drift — slow equilibration

Every model shows volume creeping upward for a long time. GRACE example over 200 frames:

```
Frame 0: 1882.42 Å³
Frame 100: 1895.84 Å³
Frame 199: 1907.55 Å³ ← still rising, ~1.3% and not converged
```

Not a bug — the system genuinely hasn't equilibrated. Needs 200k+ equilibration steps. Compounded by PBE-trained models wanting a larger cell than experiment.

● `mace-gpu` CUDA mismatch

torch `2.11.0+cu130` vs cluster CUDA 12.6. MACE-OFF23 and PolarMACE work, MACE-MPA-0 does not. Use `mace_mp0_env` for the MP/MPA/MH families.

● Dashboard shows a hardcoded paper count

`Dashboard.tsx` prints `36`; the API returns 63. Read the real value.

● Papers with no title

Some `/api/papers` entries have `title: null`, rendering as "Title not recorded". This is a **data extraction** problem in the backend, not a display bug. Count is surfaced in the Papers page header — worth investigating at the source.

10. Data layout and schema

On-disk layout

```

Piezo-LLM/
├─ data/PMC-001/
│   ├── PMC-001-gamma-glycine.cif      # structure
│   ├── PMC-001-gamma-glycine.pdf      # source paper
│   ├── PMC-001.json                   # per-crystal metadata (detailed)
│   └─ PMC-001.txt                     # RAG text summary
├─ gui/public/
│   ├── data/                          # copy of data/ served to the browser
│   └─ database/master_database.json    # summary index
├─ src/                                # python scripts
├─ simulations/
│   ├── ase-simulations/ase-nvt/... , ase-npt/...
│   └─ lammps-simulations/...
└─ vectordb/                           # Chroma store

```

⚠ `gui/public/data` is a **real directory**, not a symlink. Files added to `data/` will **not** appear in the GUI automatically — copy them, or replace it with a genuine symlink. Do **not** create `ln -s ../data gui/public/data` while that directory already exists — it produces `gui/public/data/data → ...` and Vite dies with `ELOOP: too many symbolic links`.

● **`master_database.json` field names — read this before writing UI code**

Actual keys per crystal:

```

pmc_id, molecule_name, synonyms, chemical_formula, molecular_weight,
crystal_type, component_count, ccdc_number, csd_refcode,
space_group_symbol, space_group_number, crystal_system, centrosymmetric,
is_piezoelectric, is_ferroelectric, is_pyroelectric,
property_ref_doi, structure_ref_doi,
json_schema_version, json_path, pdf_path, cif_path, txt_path,
aliases, search_text, status, validated, last_updated

```

Mismatches that have already caused bugs:

Code expected	Data actually has	Consequence
<code>cif_file_name</code>	<code>cif_path</code>	3D viewer couldn't find the CIF
<code>structure_doi</code>	<code>structure_ref_doi</code>	DOI link never rendered
<code>cell_a ... cell_gamma</code>	<i>(absent)</i>	Unit Cell card renders all dashes
<code>density_g_cm3</code> , <code>temperature_k</code> , <code>radiation</code> , <code>colour</code> , <code>habit</code>	<i>(absent)</i>	Experimental card empty
<code>cod_code</code> , <code>intermolecular_interactions</code> , <code>property_symmetry_compatible</code>	<i>(absent)</i>	conditional blocks never show

`gui/gui_src/pages/crystals/Crystal.ts` still declares the **wrong** names, which is precisely why TypeScript never caught this. **Fix the type to match the data.**

Path values are relative

```
json

"cif_path": "../data/PMC-001/PMC-001-gamma-glycine.cif"
```

Not usable as a URL directly. Strip to the filename and rebuild:

```
ts

function fileUrl(pmcId: string, storedPath: unknown, fallback: string): string
  const name = typeof storedPath === "string" && storedPath.trim()
    ? storedPath.split("/").pop()! : fallback;
  return `/data/${pmcId}/${encodeURIComponent(name)}`;
}
```

`encodeURIComponent` is required — one file is named `N-acetyl-L-lysine (L-Ack).cif`, and unencoded spaces/parentheses break the fetch.

Detailed fields live in the per-crystal JSON

`master_database.json` is a **summary**. Unit-cell parameters and experimental details are in `data/PMC-XXX/PMC-XXX.json`. `CrystalDetails.tsx` now merges both:

```
ts

const base = summaryEntry;
const detail = await fetch(`/data/${id}/${id}.json`).then(r => r.json());
setCrystal({ ...base, ...detail }); // detail wins
```

Still unverified: whether the per-crystal JSON actually contains `cell_a`, `density_g_cm3`, etc. Check with:

```
bash
```

```
python3 -c "import json; print(list(json.load(open('data/PMC-001/PMC-001.json'))))"
```

If absent, either populate them from the CIF or remove those two cards.

11. Frontend (GUI)

Stack

- **Vite 8.1** + React + TypeScript
- **Tailwind CSS**
- **react-router-dom**
- **lucide-react** (icons)
- **three.js** (3D crystal viewer)
- Dev server: `http://localhost:5173`

Structure

```

gui/gui_src/
├─ App.tsx                                # routes
├─ main.tsx
├─ components/
│   └─ layout/
│       ├── Layout.tsx                    # sidebar + navbar + <Outlet/>
│       ├── Sidebar.tsx                   # colour-coded nav
│       └─ Navbar.tsx                     # back button, global search, health bell
│   └─ ui/Card.tsx
└─ pages/
    ├── dashboard/Dashboard.tsx
    ├── Chat/AiChat.tsx                   # ← the real chat (chat.tsx is a stub)
    ├── crystals/
    │   ├── CrystalExplorer.tsx
    │   ├── CrystalCard.tsx
    │   └─ Crystal.ts                     # ⚠ type does not match the data
    ├── details/
    │   ├── CrystalDetails.tsx             # all-in-one page
    │   └─ components/Crystal3DViewer.tsx
    ├── database/
    │   ├── Database.tsx                   # tabs + admin login
    │   ├── DatabaseAdminPanel.tsx
    │   ├── PiezoClassification.tsx
    │   └─ VectorDBViewer.tsx
    ├── papers/Papers.tsx
    └─ settings/Settings.tsx

```

Routes

Path	Page
<code>/</code>	Dashboard
<code>/chat</code>	AI Chat
<code>/crystals</code>	Crystal Explorer
<code>/crystal/:id</code>	Crystal Details
<code>/papers</code>	Papers
<code>/database</code>	Database (4 tabs)
<code>/settings</code>	Settings

⚠ App.tsx must import the real chat


```
tsx

import Chat from "../pages/Chat/AiChat"; // ✅ full RAG chat
// not "../pages/Chat/chat"             // ❌ placeholder stub
```

Design system (applied during the redesign)

Element	Convention
Page background	● #080B12
Card background	● #0d1117
Borders	border-white/[0.06], hover [0.12]
Section labels	10px, uppercase, tracking-wider , text-gray-600
Accent	cyan for primary/piezoelectric
Cards	rounded-xl / rounded-2xl , hover scale-[1.02]
Icons	lucide-react, 3.5-5 px in badges

Crystal system colour code (left border + badge on cards, also used in details):

System	Colour
Trigonal	cyan
Monoclinic	blue
Orthorhombic	violet
Tetragonal	emerald
Hexagonal	amber
Cubic	rose
Triclinic	pink

Sidebar accents: Dashboard cyan · AI Chat violet · Crystal Explorer emerald · Papers amber · Database blue · Settings rose.

Piezoelectric vs non-piezoelectric — piezo cards get a coloured gradient, white text, ⚡

Zap

 badge; non-piezo get flat grey background, muted text, ⚡/

ZapOff

 badge in dim red.

Crystal 3D viewer — capabilities

Rewritten to approach Mercury/CCDC behaviour:

- **Symmetry expansion** — parses `(_symmetry_equiv_pos_as_xyz)` / `(_space_group_symop_operation_xyz)`, applies every operator, wraps fractional coords to $[0,1)$, deduplicates. *Without this you only see the asymmetric unit* — for $P3_1$ γ-glycine that's $\frac{1}{3}$ of the cell.
- **Packing modes** — asymmetric unit / unit cell / supercell (1-4 per axis)
- **Covalent-radius bonding** — `(d < r1 + r2 + tol)` instead of a flat cutoff, so C-H bonds but N...O doesn't
- **Hydrogen bonds** — dashed cyan; H bonded to N/O/F, acceptor N/O/F at 1.4-2.6 Å, D-H...A angle > ~120°
- **Cell axes** — a red / b green / c blue with sprite labels
- **Show/hide hydrogens**, atom size, bond tolerance sliders
- **CIF esd stripping** — `(5.1054(4))` → `(5.1054)`
- Spatial hash for bonds so supercells stay responsive

⚠ H-bond detection is $O(N^2)$. Fine at ~700 atoms ($2 \times 2 \times 2$ glycine); $4 \times 4 \times 4$ on a larger molecule will stutter. Untick the option or move it onto the spatial hash.

Auto-loading the CIF — `CrystalDetails` fetches it and passes `cifContent`:

```
ts

const candidates = [
  fileUrl(id!, (crystal as any)?.cif_path, `${id}.cif`),
  `/data/${id}/${id}.cif`,
];
for (const path of candidates) {
  const resp = await fetch(path);
  if (resp.ok) {
    const text = await resp.text();
    if (text.includes('_atom_site')) { setCifContent(text); return; }
  }
}
```

Navbar features

- **Back button** on every page except Dashboard
- **Global search** — crystals (PMC ID, name, formula) + papers (title, journal, DOI); `(Ctrl/⌘+K)` to open, `(↑↓)` navigate, `(Enter)` open, `(Esc)` close
- **Health bell** — real checks (backend reachable, LLM loaded, crystal DB loaded, unnamed crystals, untitled papers). Dot appears only on warning/error.

Security note

`Database.tsx` hardcodes:

```
tsx

const ADMIN_ID = 'pravega2';
const ADMIN_PASS = 'kb8DU2rG';
```

These ship to the browser and are readable in DevTools → Sources. Acceptable for a local-only tool; **must move to the backend** before any hosting. Check whether it's already in git history:

```
bash

git log --all -S "kb8DU2rG" --oneline
```

Also add `.trim()` to the comparison — autofill whitespace has already caused a false "Invalid credentials".

12. Backend (Flask API)

Runs at `http://localhost:5000`.

Endpoint	Method	Purpose
<code>/api/llm-status</code>	GET	<code>{installed: bool}</code> — is Qwen3 available
<code>/api/chat</code>	POST	RAG query → <code>{answer, sources, mode, chunks_retrieved}</code>
<code>/api/chat/test-llm</code>	POST	<code>{connected, qwen3_loaded, message}</code>
<code>/api/papers</code>	GET	<code>{papers: [...]}</code>
<code>/api/next-pmc-id</code>	GET	next free PMC ID for upload
<code>/api/upload-crystal</code>	POST	multipart PDF + CIF → creates a new PMC entry
<code>/api/admin/database-status</code>	GET	totals, complete/incomplete, piezo/ferro counts
<code>/api/admin/crystals</code>	GET	per-crystal file presence (<code>json/pdf/cif/txt</code>)
<code>/api/admin/rebuild-database</code>	POST	full scan → rebuild <code>master_database.json</code>
<code>/api/admin/rebuild-vectordb</code>	POST	re-embed all crystals into Chroma

`/api/chat` request

```
json

{
  "question": "...",
  "temperature": 0.2,
  "top_k": 5,
  "llm_url": "http://localhost:11434"
}
```

Settings come from `localStorage['piezo_settings']`, written by the Settings page.

13. LLM / RAG pipeline

```
User question
  → embedding model
  → Chroma vector search over crystal text summaries (top_k, default 5)
  → retrieved chunks + question → Qwen3-8B (Ollama, :11434)
  → answer + source PMC IDs with similarity scores
```

Piece	Value
LLM	Qwen3-8B via Ollama
LLM endpoint	<code>http://localhost:11434</code>
Vector store	Chroma (<code>vectordb/chroma.sqlite3</code>)
Indexed text	the <code>text</code> field of each <code>PMC-XXX.json</code>
Default temperature	0.2
Default top_k	5

Start Ollama:

```
bash

ollama serve
ollama pull qwen3:8b
```

Rebuild embeddings after adding or editing crystal data — Database → Administration → *Rebuild Vector Database*. It adds new / updates existing without duplicates, and skips empty placeholder folders.

Important: RAG answer quality is bounded by JSON quality. The missing fields in §10 are also missing from the embedded text, so the chat cannot answer questions about unit cells or densities that were never recorded. **Fix the data before tuning the prompt.**

14. Git repository hygiene

Repo: `https://github.com/Him3shRana/Piezo-LLM-IIT-D.git`

Standard push

```
bash
cd ~/Documents/Piezo-LLM
git status
git add .
git commit -m "message"
git push
```

⚠ Things that got committed but probably shouldn't have

The last push (15.9 MiB, 311 objects) included regenerable outputs:

- `vectordb/chroma.sqlite3`, `data_level0.bin`, `link_lists.bin`
- `vectordb_export.csv`
- `*.lampoostrj`, `*.restart`, `traj_raw.lampoostrj`
- dozens of `rdf_*.png`

Git stores **every version of a binary in full**. Re-running a simulation and re-committing `chroma.sqlite3` stores an entire new copy each time.

Recommended `.gitignore`

```
gitignore

# environments
venv-finetune/
venv-serve/
gui/node_modules/

# models
Qwen3-8B/
*.model
*.ckpt
*.pt
*.pth

# simulation outputs
runs/
300K/
*.extxyz
*.traj
*.lammprj
*.restart

# vector store
vectordb/
vectordb_export.csv

# python
__pycache__/
*.pyc
```

Stop tracking without deleting local files:

```
bash

git rm -r --cached vectordb vectordb_export.csv
git commit -m "Stop tracking regenerable outputs"
git push
```

Worth versioning vs not

Keep in git	Leave out
<code>state.json</code> , <code>config.yaml</code>	raw trajectories
RDF plots (small, are results)	<code>.restart</code> files
scripts, GUI source	Chroma DB
<code>master_database.json</code>	model weights

15. Roadmap

Phase 1 — Data integrity ● *do first*

Everything downstream depends on it, RAG included.

- ☐ Fix `Crystal.ts` to match the real schema (`cif_path`, `structure_ref_doi`, drop `cif_file_name`)
- ☐ Verify whether per-crystal JSONs contain `cell_a`, `density_g_cm3`, etc.
- ☐ Populate missing fields from CIFs, or remove the empty cards
- ☐ Investigate papers with `title: null` at the extraction step
- ☐ Fix the hardcoded `36` on the Dashboard
- ☐ Rebuild vector DB after the data is corrected

Phase 2 — Simulations page ● *highest value*

Your research output is currently invisible in the GUI.

- ☐ Run matrix: crystal × model × conditions, with live status from `state.json`
- ☐ **Model comparison table** — simulated vs experimental density per model (*this is the core deliverable*)
- ☐ Volume / temperature plots parsed from stage logs
- ☐ RDF viewer using the existing `rdf_*.png` and `rdf_compare/`
- ☐ Timing comparison from `calculate_total_time()`

Data access decision: start by symlinking `simulations/` into `gui/public/` and reading `state.json` directly from React. Swapping to a Flask `/api/simulations` endpoint later is a one-line fetch change — and that same endpoint becomes the SSH reader in Phase 3.

Phase 3 — Cluster connection

Browser → Flask (laptop) → SSH → HPC

A browser cannot SSH; Flask must broker it.

- **3a. Sync-then-browse** — `(rsync)` results down, GUI reads local files. *Do this first* — 80% of the value, no auth complexity.

bash

```
rsync -avz cyz218376@pragya.iitd.ac.in:~/himesh_work/testing/runs/ ~/Document
```

- **3b. Live status** — `(paramiko)` in Flask, `(cat state.json)` / `(qstat)` on demand. Needs passwordless SSH keys.
- **3c. Job submission** — generate + `(qsub)` PBS scripts, poll `(qstat)`. The agentic layer.

⚠ Never put SSH keys in the React app. Credentials live in Flask. Poll `(qstat)` on demand or every 30-60 s — not continuously, out of respect for the login node.

Phase 4 — AI Chat depth

Once simulation data is queryable, the chat can answer "*which model gave density closest to experiment for PMC-001?*" — which is far more valuable than a nicer chat UI.

Phase 5 — Remaining models

- ☐ Integrate EquiformerV2 (`(eqv2_86M_omat_mp_salex.pt)` — already downloaded)
- ☐ Integrate eSEN-30M-OAM (`(esen_30m_oam.pt)` — already downloaded)
- ☐ Install NequIP-OAM-XL

16. Pending checklist

Simulation

- ☐ Resolve ORB / MatterSim large-stress NPT problem
- ☐ Decide final `(taup)` / `(compressibility)` per model, benchmarked against experiment
- ☐ Run the full model × crystal matrix
- ☐ Confirm PET works end-to-end on a compute node with `(HF_HUB_OFFLINE=1)`
- ☐ Compare γ -glycine packing in the 3D viewer against Mercury (validates symmetry expansion)
- ☐ Consider `(dispersion=True)` for MACE-MP-0 on molecular crystals

Code quality

- ☐ Fix `Crystal.ts` type definitions
- ☐ Move admin credentials out of the frontend
- ☐ Add `.trim()` to the login comparison
- ☐ Clear the 3 unused-import warnings (Dashboard, DatabaseAdminPanel, CrystalDetails)
- ☐ Delete obsolete detail sub-components (`HeaderCard`, `ChemicalInfoCard`, `StructureCard`, `DownloadCard`, `ViewerCard`, `UnitCellCard`, `BasicInfoCard`, `PropertyCard`, `DescriptionCard`) — all folded into `CrystalDetails.tsx`
- ☐ Delete `SearchBar.tsx` / `FilterPanel.tsx` — folded into `CrystalExplorer.tsx`
- ☐ Delete `chat.tsx` stub once `AiChat` is confirmed wired
- ☐ Update `.gitignore` and untrack regenerable outputs

Infrastructure

- ☐ Decide whether `gui/public/data` should be a real symlink to `data/`
 - ☐ Set up passwordless SSH if going for live cluster status
 - ☐ Write PBS job scripts for long production runs
-

17. Command cheat sheet

Run a simulation

```

bash

# CHGNet
module load cuda/12.6 gcc/10.5.0 conda-2025/pytorch-gpu-2.9.1/py3.14
source ~/home/software/chgnet_env/bin/activate
python all_npt_ase.py --pmc PMC-001 --model chgnet --version pretrained \
  --temperature 300 --pressure 1.0 --equil-step 200000 --target-step 2000000 --

# MACE-OFF23 (best for organic)
source ~/home/software/mace-gpu/bin/activate
python all_npt_ase.py --pmc PMC-001 --model mace-off23 --version medium ...

# MACE-MPA-0 / MH-1
source ~/home/software/mace_mp0_env/bin/activate
python all_npt_ase.py --pmc PMC-001 --model mace-mpa0 --version medium ...
python all_npt_ase.py --pmc PMC-001 --model mace-mh1 --version spice ...

# GRACE
module load cuda/12.6 gcc/10.5.0 conda-2025/tf-gpu-2.19.1/py3.12
source ~/home/software/grace_env/bin/activate
python all_npt_ase.py --pmc PMC-001 --model grace --version GRACE-2L-OAM ...

# ORB (conservative variant for NPT!)
source ~/home/software/orb_env/bin/activate
python all_npt_ase.py --pmc PMC-001 --model orb \
  --version orb-v3-conservative-inf-omat ...

# MatterSim
source ~/home/software/mattersim_env/bin/activate
python all_npt_ase.py --pmc PMC-001 --model mattersim --version 5M ...

# UMA (trained on molecular crystals)
source ~/home/software/uma_env/bin/activate
python all_npt_ase.py --pmc PMC-001 --model uma --version uma-m-1p1 ...

# PET
source ~/home/software/upet_env/bin/activate
export HF_HUB_OFFLINE=1
python all_npt_ase.py --pmc PMC-001 --model pet --version pet-oam-xl ...

```

List registered models

```

bash

python all_npt_ase.py --list-models

```

Monitor a running job

```
bash
```

```
cat runs/PMC-001/ase-npt/<model>/300K_1bar_2x2x2/production/state.json  
tail -f runs/PMC-001/ase-npt/<model>/300K_1bar_2x2x2/production/production.log  
nvidia-smi
```

Inspect a trajectory

```
python
```

```
from ase.io import read  
frames = read("production/traj.extxyz", index=":")  
for i, a in enumerate(frames):  
    print(f"{i}: V={a.get_volume():.4f} Å³ cell={a.get_cell().cellpar()[ :3]}")
```

Download a model (login node only)

```
bash
```

```
# MACE family
```

```
python -c "from mace.calculators import mace_mp; mace_mp(model='medium-mpa-0')"  
python -c "from mace.calculators import mace_mp; mace_mp(model='mh-1')"
```

```
# ORB
```

```
python -c "from orb_models.forcefield import pretrained; pretrained.orb_v3_cons"
```

```
# MatterSim
```

```
python -c "from mattersim.forcefield import Potential; Potential.from_checkpoint"
```

```
# UMA
```

```
python -c "from fairchem.core import pretrained_mlip; pretrained_mlip.get_predi"
```

```
# GRACE
```

```
grace_models download GRACE-2L-OAM
```

```
# manual .model file
```

```
wget -O ~/himesh_work/mace_models/mace-mpa-0-medium.model \  
"https://github.com/ACEsuit/mace-mp/releases/download/mace_mpa_0/mace-mpa-0-m"
```

GUI

```
bash
```

```
cd ~/Documents/Piezo-LLM/gui  
npm run dev # http://localhost:5173
```

Backend + LLM

```
bash
```

```
cd ~/Documents/Piezo-LLM/src
```

```
python backend.py          # http://localhost:5000
```

```
ollama serve                # http://localhost:11434
```

Transfer files

```
bash
```

```
# push a script up
```

```
scp all_npt_ase.py cyz218376@pragya.iitd.ac.in:~/himesh_work/testing/
```

```
# pull results down
```

```
rsync -avz cyz218376@pragya.iitd.ac.in:~/himesh_work/testing/runs/ ./runs/
```

18. Troubleshooting table

Symptom	Cause	Fix
<code>Atoms object has no calculator</code>	<code>CHGNetASECalc</code> defined inside a function, or calculator not rebuilt on resume	move class to module level; rebuild calculator on resume (§8.1, §8.3)
<code>UnboundLocalError: CHGNetASECalc</code>	duplicate class definition inside <code>build_calculator</code> shadowing the global	delete the inner copy
~8.5 s/step (should be ~0.14)	<code>_safe_refresh</code> doubling force calls	remove the patch (§8.2)
<code>nVRTC: failed to open libnVRTC-builtins.so.13.0</code>	torch cu130 vs cluster CUDA 12.6	use <code>float32</code> , run from <code>mace_mp0_env</code>
<code>InductorError: IndexError: list index out of range</code>	ORB <code>torch.compile</code>	pass <code>compile=False</code>
<code>Head keyword was not provided</code>	MACE-MH-1 is multi-head	pass <code>head="spice_wB97M"</code>
<code>Tunnel connection failed: 302) (Network is unreachable)</code>	compute node has no internet	download on login node first
PET/UMA hangs or errors on compute node	HuggingFace checking for updates online	<code>export HF_HUB_OFFLINE=1</code>
NPT volume perfectly constant	direct ORB model returns no stress	use <code>orb-v3-conservative-*</code>
Volume rising and never settling	not equilibrated / PBE overestimates cell	equilibrate 200k+ steps; check density vs experiment
<code>Another run is already in progress</code>	stale <code>.run.lock</code>	delete the lock file
<code>ELoop: too many symbolic links</code>	recursive symlink <code>public/data/data</code>	<code>rm gui/public/data/data</code> (no trailing slash, no <code>-rf</code>)
3D viewer opens empty	CIF not found / not served	check <code>cif_path</code> , ensure the file is under <code>gui/public/data/<PMC>/</code>
3D structure looks sparse, bonds broken	only the asymmetric unit is shown	symmetry expansion — set Packing to "Unit cell"

Download buttons 404	links point to <code>/database/cif/...</code>	use <code>/data/{id}/{filename}</code> from <code>cif_path</code>
"Invalid credentials" with correct password	autofill whitespace, or <code>pragya2</code> vs <code>pravega2</code>	add <code>.trim()</code> ; retype the ID
Brown dots in VS Code file tree	unused imports (warnings, not errors)	open PROBLEMS panel, remove the imports

Quick reference — the five things most easily forgotten

1. **Download models on the login node.** Compute nodes have no internet.
2. **ORB direct models cannot do NPT.** Use `orb-v3-conservative-inf-omat`.
3. **Never re-add `_safe_refresh`.** It makes simulations ~60× slower.
4. **The data field is `cif_path`, not `cif_file_name`.** And `structure_ref_doi`, not `structure_doi`.
5. **Back up before `--fresh`.** It overwrites trajectories irrecoverably.

Maintained for the Piezo-LLM project · IIT Delhi · Department of Chemistry